Retrieval-Augmented Question and Answer (RAG) (Using Small Language Models)

Question 1

Designing a text assistant

Problem: Design a system that, upon receiving a linguistic query, first retrieves relevant documents from a small set and then generates the final answer using a small linguistic model.

Dataset:

This dataset contains the question - In this exercise, the standard WikiQA dataset is used and includes the question text, reference answers, and candidate sentences from Wikipedia documents. Text-based answers are based on the answers. You can use the cli-huggingface tool to access this library . To access the information in this dataset, check out the following link. https://huggingface.co/datasets/microsoft/wiki_qa

1 - **Implementation of the RAG base system:**

Implement a RAG system including the following components:

– Transformer -based text embedding model (such as MiniLM or 5E)

– Indexing documents using FAISS

– A small language model for response generation (such as 5T-FLAN)

The overall system architecture should include the following steps:

1 - Receiving a question

2 - Calculating the Question Embedding

3 - Retrieving the top k documents

Question 1 continued on the *next page...*

4 - Attaching recovered documents to Prompt

5 - Generating the final answer

2 - **Analysis of the text embedding module:**

Examine the effect of choosing an adaptive model:

Transformer – Comparison of at least two Embedding models based on

– Investigating the effect of embedding size

– Qualitative analysis of retrieved documents

The main criteria in this section:

Recall@k –

3 - **Comparison of Small Language Models (SLM):**

In this section, we compare the performance of the RAG system using the following three small language models:

flan-t5-small –

t5-small –

distilgpt2 –

For each model:

– The quality of the final response should be evaluated.

– Report the response time on the CPU .

– Analyze the stylistic and conceptual differences in the responses.

4 - **Adding fine-tuning of weight (LoRA / PEFT):**

Select the best SLM model from the previous question and retrain it on the dataset using fine-tuning techniques. All decisions made in designing the tuning method [1] Record the exact weightings you made in your report file and explain the reason for each decision. Describe the impact of these decisions on the model's performance by re-running the evaluation on the model (once per

Parameter-Efficient Fine-Tuning 1

Question 1 continued on the *next page...*

(Separating each decision once, considering all decisions). Note that to save time, you can use a small portion of the data set (between 100 and 500 question-answers) at this stage.

5 - **Sensitivity analysis of parameters:**

Examine the effect of changing the following parameters:

– Number of retrieved documents (k)

– Prompt length

– Production temperature (Temperature)

The results should be presented in the form of a table or graph.

6 - **Conceptual analysis:** Based on the experiments performed, present your analysis on the following items in the report file:

Mention it.

– The role of embedding quality in RAG architecture

– Challenges of running RAG on CPU

– System improvement paths

1 - Implementation is done with Python and PyTorch .

2 - The use of GPU is not allowed.

and peft are allowed. 3 - transformers, transformers-sentence libraries

4 - The final report should include analysis, tables, and graphs.

5 - The final file should be sent with the name zip05.HW_StudentID .

6- Any copying will result in the elimination of all scores.